DJANGO FRAMEWORK

Django is a free, open-source, high-level Python web framework used to build web applications.

It is built using Python and helps developers create web applications quickly, securely, and in an organized manner.

Django was created by **Adrian Holovaty** and **Simon Willison**.

Django follows the **MVT architecture**, which stands for:

- Model
- View
- Template

Django provides many built-in features, so developers do not have to write everything from scratch.

SHREEHARI N

Important Features of Django

- Django is based on Python.
- It follows the MVT architecture.
- It provides a built-in admin panel.
- It supports database operations using ORM.
- It provides built-in authentication.
- It supports template rendering.
- It provides good security features.
- It is suitable for small and large web applications.
- It helps in faster development.

Interview Point

Django is called a high-level framework because it provides many ready-made features required for web development.

DJANGO MVT ARCHITECTURE

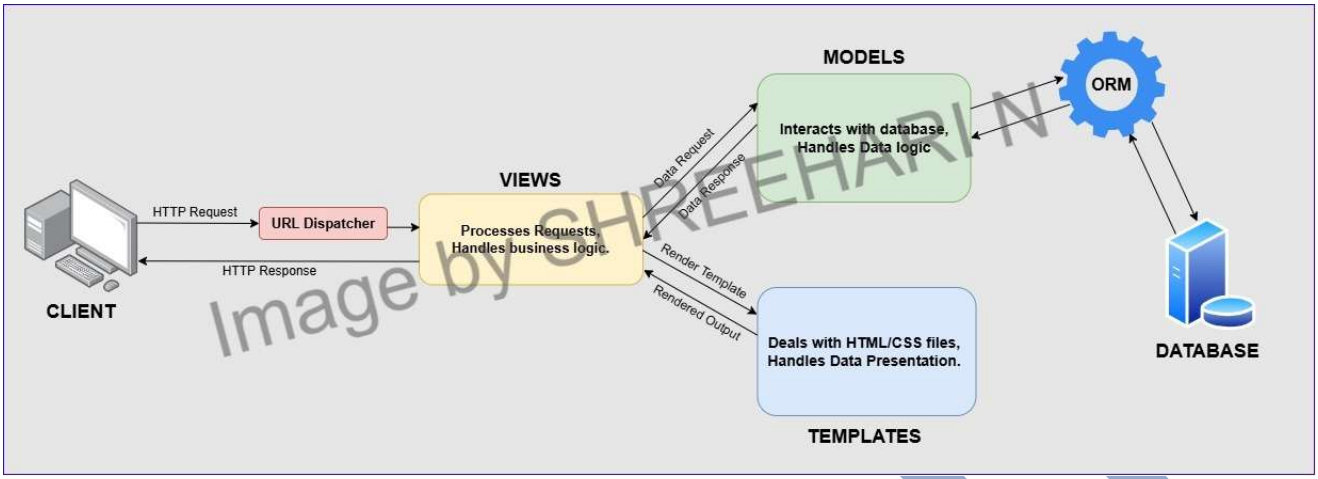


Image by SHREEHARI N

Django follows the **MVT architecture**. MVT stands for:

- Model
- View
- Template

MVT separates the application into different parts. Each part has a specific responsibility.

Model

The Model represents the data layer of the application.

It is responsible for:

- Defining database tables.
- Defining fields or columns.
- Handling data-related logic.
- Working with the database using Django ORM.

View

The View represents the business logic layer of the application.

It is responsible for:

- Receiving requests.
- Processing logic.
- Communicating with models.
- Sending data to templates.
- Returning responses.

SHREEHARI N

Template

The Template represents the presentation layer of the application.

It is responsible for:

- Displaying data to the user.
- Writing HTML pages.
- Showing dynamic data using Django Template Language.

Django Installation

Before creating a Django project, Django must be installed. It is recommended to install Django inside a virtual environment.

Command to install Django

```
pip install django
```

Command to check django version

```
import django
django.get_version()
```

Django Project Creation

A Django project represents the complete web application. It contains the main settings and configuration files required to run the application.

A project can contain multiple applications.

Example :

Project : School Management System

```
Apps :
    - students
    - teachers
    - courses
    - attendance
```

SHREEHARI N

django-admin

django-admin is a command-line utility provided by Django. It is mainly used to create Django projects.

Command to create a project

```
django-admin startproject <project_name>
```

Command to run server

```
python manage.py runserver
```

Command to run server on different port

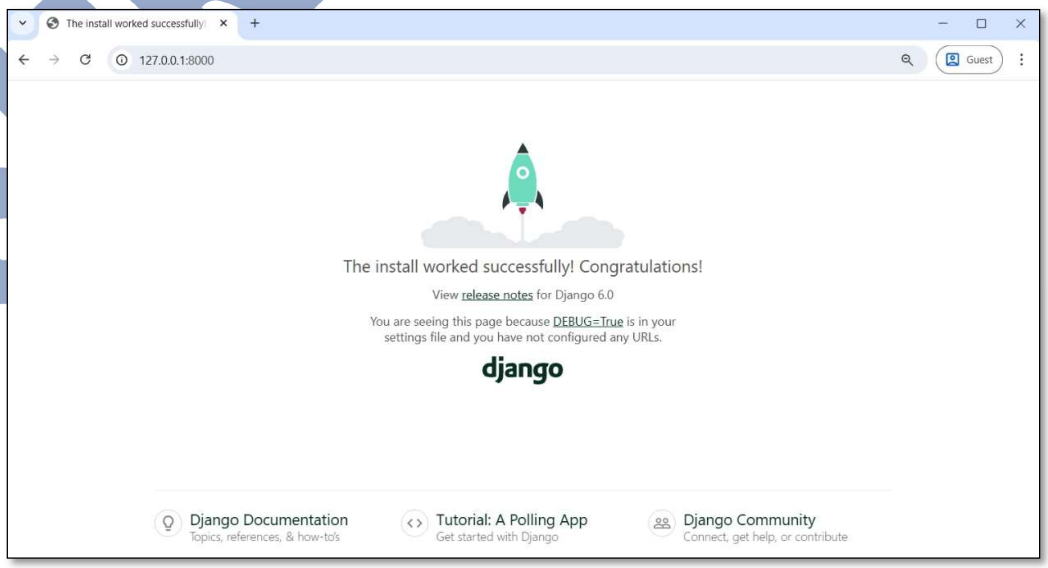
```
python manage.py runserver <port_number>
```

Steps to Create and Run a Project

```
django-admin startproject myfirstproject
cd myfirstproject
python manage.py runserver
```

By default, django runs on port **8000**. The project can be opened in the browser using http://127.0.0.1:8000 or http://localhost:8000.

Open a browser and access the server URL shown. If the default Django success page appears, the project setup is successful.

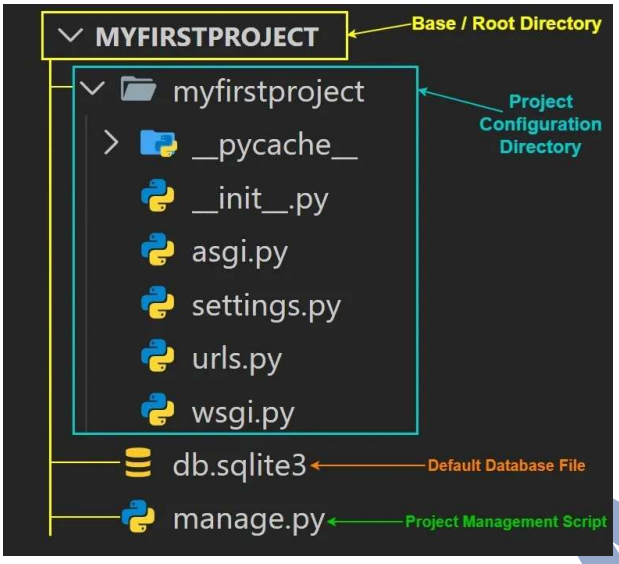


To stop the server: Ctrl + C

SHREEHARI N

Django Project Directory Structure

After creating a Django project, Django automatically creates a standard directory structure.



Important files in a Django Project

1. **settings.py :** contains important project configurations such as :
    - Installed apps
    - Database settings
    - Middleware
    - Templates
    - Static files
    - Security settings
2. **urls.py :** contains URL patterns. It maps URLs to views.
3. **wsgi.py :** WSGI (Web Server Gateway Interface) describes how a web server communicates with the web application, and how web applications can be chained together to process a request. It provides standard for **synchronous python apps**.
4. **asgi.py :** ASGI (Asynchronous Server Gateway Interface) is intended to provide standard interface between async-capable python web servers, frameworks and applications. Basically it provides standards for both asynchronous and synchronous apps
5. **manage.py :** manage.py is a command-line utility automatically created in every Django project. It is used to run project-level commands such as running the server, creating apps and applying migrations.

SHREEHARI N

6. **db.sqlite3 :** is the default database file created by Django. It is mainly used for learning and development.

Django Application

A Django application is a smaller module inside a Django project. An application is created to handle one specific feature or functionality.

A single Django project can contain multiple applications.

Example : E-Commerce Website

```
Apps :

    - users
    - products
    - cart
    - orders
    - payments
```

Need for Django Applications

Applications are used because :

- They divide a large project into smaller parts.
- They keep related code together.
- They make the project easy to maintain.
- They improve code reusability.
- They allow multiple developers to work on different features.
- They make the project more organized.

Command to create an application

```
python manage.py startapp <app_name>
```

SHREEHARI N

Project vs Application

Django Project	Django Application
It is the complete web application.	It is a feature/module inside the project.
It contains project-level settings.	It contains feature-specific code.
One Project can have many apps.	One app usually handles one feature.

Registering a Django Application

After creating an application, it must be registered in **settings.py**. Apps are registered inside the **INSTALLED_APPS** list.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'app1',
]
```

Why App registration is required

App registration is required because :

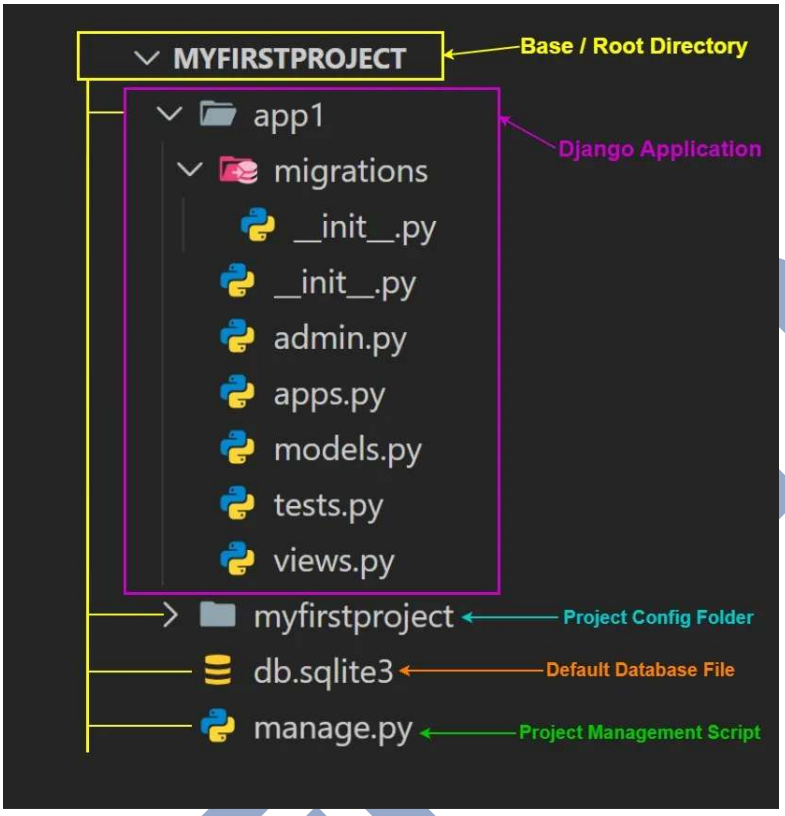
- Django should know that the app is part of the project.
- Django can load the app properly.
- Models inside the app become active.
- Migrations can be created.
- Admin configurations can work.

If an app is not registered, Django will not recognize it completely.

SHREEHARI N

Django Application Directory Structure

After creating an app, Django creates the following files and folders :



Important files in a Django App

1. **admin.py** : Used to register models with Django admin.
2. **apps.py** : Used to configure the Django application. It helps django recognize and manage the application.
3. **models.py** : Used to define data models for application. Models are written as Python classes. Models represent the ***data layer*** of the application.
4. **tests.py** : Used to write ***test cases*** for the application.
5. **views.py** : Used to define ***views*** for the application. It handles the incoming requests and returns responses. A significant part of ***business logic*** is written here.
6. **Migrations folder** : Stores the migration files created based on model changes.

SHREEHARI N

DJANGO VIEWS

A view is a Python function or class that receives a web request and returns a web response.

Views are usually written inside the **views.py** file of an application.

A View can return :

- Plain text response
- HTML response
- Rendered template
- Redirect response
- JSON response

```python
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to django views")
```

Types of Views

Function-Based views :

A Function-based view is a python function that accepts an HTTP request object and returns an HTTP response. It provides direct and explicit way to define application logic in Django.

Function based views are commonly used for simple request-response handling where control flow needs to be clearly visible.

Class-Based views :

A Class-Based View is implemented using a python class, where different request types are handled using class methods such as **get( )** and **post( )**.

Class-based views provide a structured way to organize view logic, especially when handling multiple HTTP request types for same URL.

SHREEHARI N

URL ROUTING

URL routing is the process of mapping a URL to the appropriate view. Django uses **urls.py** to determine which view should handle a given request.

urls.py

urls.py is the URL configuration file of a Django project or application. It contains a list of URL patterns stored in a variable named **urlpatterns**.

For every incoming request, Django extracts the path part of the URL, checks it against the pattern in **urlpatterns** and selects the first matching pattern. Once a match is found, the associated view is executed. If no pattern matches, Django returns a ***404 Not Found*** response.

```
from django.urls import path
from . import views


urlpatterns = [
    path('', views.home),
    path('about/', views.about),
    path('contact/', views.contact)
]
```

path( )

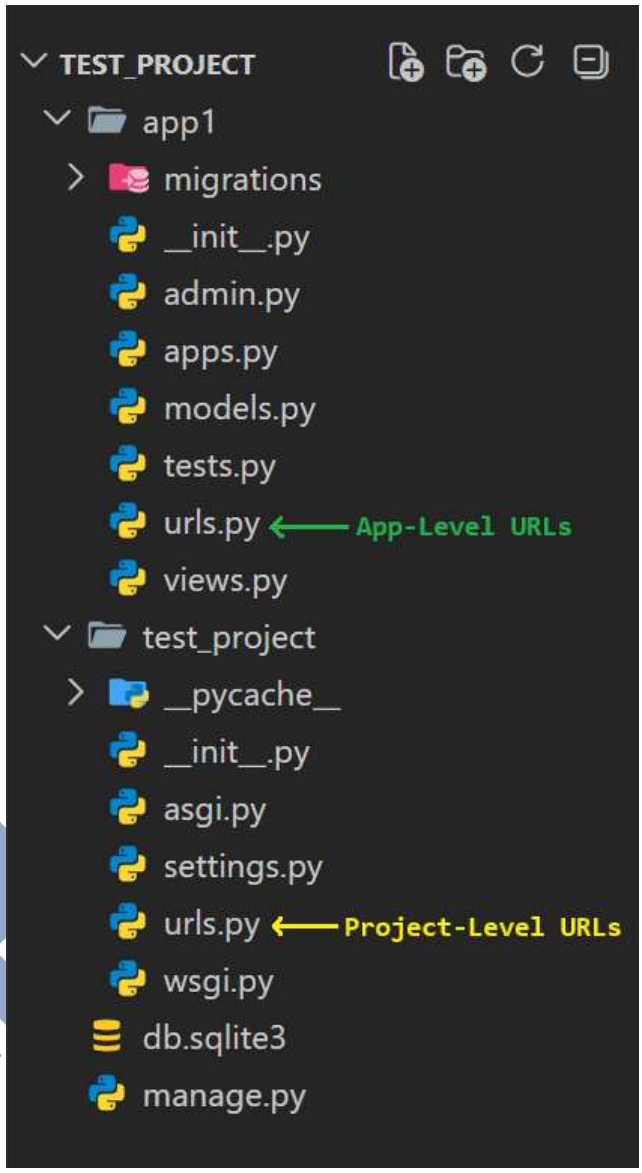
It is a High-Order Function used to define a URL pattern in Django. It maps a URL route to a view that should handle requests for that route.

```
path(route:str, view:callable, name:str)
```

- **route** – represents the URL pattern.
- **view** – represents the view function that should be executed.
- **name** – used to give a name to the URL pattern. It is optional but recommended.

In Django, URLs can be managed at two levels :

I. Project-level URLs
II. App-level URLs



Project-Level URLs

The project-level **urls.py** acts as the main entry point for URL routing. It is present inside the project configuration folder.

App-Level URLs

Each application can have its own **urls.py**. This helps keep URLs organized based on features.

The separation keeps URL handling clean, modular, and easy to maintain.

SHREE

include( )

This function is used to connect app-level URLs with project-level URLs. This helps organize URLs by feature instead of defining all routes in a single file.

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('app1/', include('app1.urls')),
]
```

Advantages of App-Level URLs

- Keeps project-level urls.py clean.
- Allows each app to manage its own URLs.
- Improves readability.
- Makes large projects easier to maintain.

SHREEHARI N